

HDR/HRM and Resonant Sparse Attention: Deterministic Integer Signatures, Resonance Retrieval, and MatMul-Free Sequence Modeling

Christian Heinrich Hohlfeld
Konstanz, Germany
christianhohlfeld.com
ORCID: 0009-0003-6634-9045

February 24, 2026

Rights. © 2026 Christian Heinrich Hohlfeld (Konstanz, Germany). **All rights reserved.** *No license is granted* to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of this work (including any accompanying software) without prior written permission from the author.

Licensing inquiries: christianhohlfeld.com ORCID: 0009-0003-6634-9045

Abstract

We formalize an end-to-end system designed primarily as a *VRAM-reduction layer* for large language models: by strictly bounding and intelligently selecting context via deterministic integer-only retrieval, it enables running large models at very low VRAM, on commodity and older hardware. We call the retrieval component *Hohlfeld Resonance Model* (HRM), grounded in the *Hohlfeld Data Representation* (HDR) method. The full pipeline consists of: (1) HDR-derived integer signatures and resonance retrieval (HRM); (2) integer Maximal Marginal Relevance (MMR) selection; (3) deterministic prompt budgeting; and (4) a local renderer (with optional custom FlashAttention kernel). Full determinism of the retrieval core is a strong additional property with formal proofs (end-to-end determinism, candidate completeness, MMR correctness). Resonant Sparse Attention, a MatMul-free transformer-style extension, is also proposed.

1 Motivation and Positioning

Large language model inference imposes two primary VRAM demands: (i) static model weights, and (ii) the key-value cache that grows with context length. Standard retrieval-augmented generation (RAG) systems typically address (ii) at most, and introduce non-determinism through probabilistic vector similarity search.

HRM FlashRenderer addresses *both* VRAM demands by a different mechanism: the retrieval core (HRM) strictly bounds the context delivered to the renderer using deterministic integer arithmetic, so the renderer itself can be a small, CPU-mapped model requiring zero VRAM by default.

Unlike traditional RAG, HRM FlashRenderer is not merely a retrieval system. It is a thin efficiency layer that enables running large models at very low VRAM by strictly bounding and intelligently selecting context. Full retrieval determinism is a strong additional property of the system, not its sole purpose.

Table 1: Comparison of VRAM-reduction techniques for large language models.

Approach	Primary Goal	VRAM Reduction	Method	Determinism
HRM FlashRenderer (Ours)	VRAM-low big model inference	Very High	Context bounding + small renderer	High (Bonus)
4-bit/8-bit Quantization	VRAM reduction	High	Weight compression	None
Layer/CPU Offloading	VRAM reduction	High	Partial model on CPU	None
RAG (standard)	Context reduction	Medium	Vector search + chunking	Low
Context Compression	Context reduction	Medium	Token compression	Low
Speculative Decoding	Speed	Low	Draft model	None

2 Scope and Non-Negotiable Design Goals

This work targets reproducible, machine-checkable behavior. The design goals are:

- **VRAM efficiency:** large models run locally with minimal GPU memory via strict context bounding.
- **Superdeterminism (retrieval core):** identical inputs yield identical retrieval outputs across runs and platforms.
- **Integer-only retrieval:** bounded integers; no floating point; no stochastic sampling in the core.
- **MatMul elimination (retrieval core):** the HRM retrieval avoids dense matrix multiplication entirely.

Remark 2.1. This paper does *not* claim that eliminating MatMul automatically yields human-level language understanding. It claims that a transformer-style *control structure* (multi-head, multi-layer, context-dependent selection) can be implemented via deterministic sparse associative operators derived from HDR, rather than via dot products and dense linear layers.

3 HDR: Deterministic Residual-Vector Encoding

HDR defines a deterministic and proportional framework for encoding an ordered dataset $D = \{d_1, d_2, \dots, d_n\}$ as a linked set of residual vectors relative to a reference vector v_0 . Each data point d_i is represented by a residual vector v_i with length (amplitude) L_i and angle (phase) θ_i :

$$v_i = L_i(\cos(\theta_i)\hat{i} + \sin(\theta_i)\hat{j}), \quad (1)$$

with angles distributed deterministically by

$$\theta_i = \frac{2\pi i}{n}. \quad (2)$$

HDR further states that the original data can be reconstructed exactly via the cumulative sum of residual vectors:

$$d_i = v_0 + \sum_{k=1}^i v_k. \quad (3)$$

Assumption 3.1 (HDR axioms). Equations (1), (2), and (3) are applied as specified in the HDR method, with a fixed reference vector v_0 (typically the zero vector) and a fixed ordering of data points.

Proposition 3.1 (Exact reconstruction). *Under Assumption 3.1, the dataset D is reconstructible exactly from $(v_0, v_1, \dots, v_n)$ by (3).*

Proof. Equation (3) provides the reconstruction rule. Because it is a deterministic cumulative sum over the residual vectors, the reconstructed d_i is uniquely determined for each i . $\square$

Proposition 3.2 (Uniqueness (HDR-level)). *Under Assumption 3.1, uniqueness follows from (i) distinct angular mapping via (2) and (ii) proportional length encoding via L_i ; therefore different ordered datasets induce distinct HDR representations.*

Proof. Distinct positions i yield distinct angles by (2). Given fixed ordering, each residual vector is determined by its pair (L_i, θ_i) ; changing either magnitude or position changes at least one residual vector, and therefore changes the representation. $\square$

4 From HDR to Discrete Resonance Keys

To eliminate MatMul in retrieval and sequence selection, we map streaming inputs to *discrete resonance keys* that are sparse by construction.

Definition 4.1 (Discrete key space). Fix $B = 2^b$ bins. The discrete phase circle is $\mathbb{Z}_B = \{0, 1, \dots, B-1\}$.

Definition 4.2 (Deterministic key extractor). A deterministic key extractor is a function

$$P : \Sigma^* \rightarrow (\mathbb{Z}_B)^*$$

that maps an input sequence to an ordered list of bin indices.

Assumption 4.1 (Fixed integer kernel). P is implemented solely via fixed, bounded integer arithmetic (e.g. fixed-window integer convolution and a deterministic bin-mapping rule), and its parameters are fixed for a given system version.

Proposition 4.1 (Key determinism). *Under Assumption 4.1, $P(x)$ is deterministic for any $x \in \Sigma^*$.*

Proof. P is a composition of deterministic integer operations with fixed parameters. $\square$

Remark 4.1 (Representation vs. projection). HDR as stated in Section 2 provides exact reconstruction at the residual-vector level. A discrete key extractor P is a *projection* used for routing/association; it may be lossy by design. This work uses HDR to justify deterministic, proportional, ordered encoding; it uses P to obtain sparse, hardware-friendly keys.

5 HRM: Resonance Retrieval by Inverted Index

Let the corpus be $D = \{d_1, \dots, d_N\} \subseteq \Sigma^*$. We use the discrete keys produced by P .

Definition 5.1 (Signature set). For an item x , define a set signature

$$S(x) := \{k \in \mathbb{Z}_B : k \text{ occurs in } P(x)\}.$$

Definition 5.2 (Posting index). Define $B : \mathbb{Z}_B \rightarrow \mathcal{P}(\{1, \dots, N\})$ by

$$B(k) := \{i : k \in S(d_i)\}.$$

Definition 5.3 (Resonance score). For query q and item d_i , define

$$R(q, i) := |S(q) \cap S(d_i)|.$$

Definition 5.4 (Candidate set).

$$C(q) := \bigcup_{k \in S(q)} B(k).$$

Lemma 5.1 (Candidate completeness). *If $R(q, i) > 0$, then $i \in C(q)$.*

Proof. $R(q, i) > 0$ implies $S(q) \cap S(d_i) \neq \emptyset$, so there exists $k \in S(q)$ with $k \in S(d_i)$. Then $i \in B(k)$, hence $i \in C(q)$. $\square$

Definition 5.5 (Deterministic Top- K). Fix a total order $\prec$ on ids (e.g. numeric ascending). Define $\text{TopK}(q)$ as the ordered list of the K ids with largest $R(q, i)$ over $i \in C(q)$, breaking ties by $\prec$.

Theorem 5.1 (Deterministic retrieval). *Under Assumptions 4.1 and fixed tie-breaking order $\prec$, $\text{TopK}(q)$ is deterministic.*

Proof. By Proposition 4.1, signatures are fixed; therefore all $R(q, i)$ are fixed. A deterministic tie-break over a total order yields a unique ranking. $\square$

6 Resonant Sparse Attention: MatMul-Free Transformer-Style Sequence Modeling

We now describe a deterministic alternative to dot-product self-attention and dense linear layers.

6.1 State construction (multi-head, multi-lag, multi-layer)

Let a streaming input be tokenized into bytes $\{0, \dots, 255\}$. A fixed HDR-derived kernel produces, at each step, one bin per head. We store the last L bins (lags) for each head and each layer.

Definition 6.1 (Layer-head state). Fix integers $\text{LAYERS}, \text{HEADS}, \text{LAGS}$. At time t , define the state key

$$K_t \in (\mathbb{Z}_B)^{\text{LAYERS} \cdot \text{HEADS} \cdot \text{LAGS}}$$

as the concatenation of all lag bins for all heads and layers.

Assumption 6.1 (State determinism). Given fixed kernel parameters and fixed token stream, the state sequence (K_t) is deterministic.

6.2 Training as set-compilation (no weights, no counts)

We *do not* estimate probabilities. We compile an exact transition relation as sets.

Definition 6.2 (Next-token relation). For each observed transition $K_t \rightarrow y_{t+1}$ (next byte), store set membership:

$$\text{NextSet}[K_t] \supseteq \{y_{t+1}\}.$$

Proposition 6.1 (Training is MatMul-free). *Training requires no dense matrix multiplication.*

Proof. Each step computes a fixed number of integer-key updates and sets a bit in a 256-bit set. No operation of the form Wx with dense W is required. $\square$

6.3 Inference by sparse associative recall

To avoid fragile exact-state lookup, we retrieve a *best matching* memorized state by overlap in discrete key space.

Definition 6.3 (Overlap score). For query key K and stored key $K^{(j)}$, define

$$\text{Overlap}(K, K^{(j)}) := \#\{p : K[p] = K^{(j)}[p]\},$$

i.e. the number of matching components.

Definition 6.4 (Deterministic best match). Choose

$$j^* := \arg \max_j (\text{Overlap}(K, K^{(j)}), \prec_{\text{state}}),$$

with fixed tie-break order $\prec_{\text{state}}$ (e.g. smallest id).

Definition 6.5 (Deterministic token choice). Let $A := \text{NextSet}[K^{(j^*)}]$. If $A = \emptyset$, the system halts. Otherwise select a token deterministically from A by a fixed function of K (e.g. select the k -th set bit where $k = \sum_p K[p] \bmod |A|$).

Theorem 6.1 (Inference is MatMul-free). *Inference requires no dense matrix multiplication.*

Proof. Computing overlaps between discrete keys can be implemented via inverted indices: for each component position p and bin value $K[p]$, consult a posting list of states that share that component. Scoring increments bounded integer counters; selection over a 256-bit set uses bitwise operations. No dense dot products or dense matrix-vector products are required. $\square$

Theorem 6.2 (End-to-end determinism). *Under Assumptions 4.1 and 6.1, and fixed tie-break rules, training yields a deterministic model; inference yields deterministic outputs for a given prompt.*

Proof. All operations are deterministic integer computations with fixed parameters and explicit tie-breaking. $\square$

6.4 Complexity (informal bounds)

Let M be the number of stored states and $P = \text{LAYERS} \cdot \text{HEADS} \cdot \text{LAGS}$.

Proposition 6.2 (Per-token inference cost). *With an inverted index over (p, bin) pairs, best-match retrieval costs $O\left(\sum_{p=1}^P |\text{Post}(p, K[p])|\right)$ counter increments, followed by $O(1)$ operations on a 256-bit set (constant 32-byte scans).*

7 Deterministic Safe Division (Auxiliary Result)

Some deterministic pipelines require normalization ratios. If a denominator can be zero, a deterministic shift-unshift procedure can be used:

Definition 7.1 (Shift-unshift division). For real numbers a, b and $\delta > 0$, define

$$\text{Div}_\delta(a, b) := \frac{a + \delta}{b + \delta} - \delta.$$

Proposition 7.1 (Ratio preservation). If $b \neq 0$, then $\text{Div}_\delta(a, b) = \frac{a}{b}$.

Proof.

$$\frac{a + \delta}{b + \delta} - \delta = \frac{a + \delta - \delta(b + \delta)}{b + \delta} = \frac{a + \delta - \delta b - \delta^2}{b + \delta} = \frac{a}{b}.$$

□

Remark 7.1. When $b = 0$, $\text{Div}_\delta(a, 0) = \frac{a+\delta}{\delta} - \delta$ is defined and deterministic for any fixed $\delta > 0$.

8 Conclusion

HDR provides a deterministic, proportional residual-vector framework with exact reconstruction and uniqueness at the representation level. By introducing a fixed integer key extractor P , we obtain sparse discrete resonance keys enabling HRM retrieval and a MatMul-free transformer-style sequence core. The resulting training and inference pipelines operate via bounded integer operations and sparse associative recall rather than dense matrix multiplication.

Acknowledgments

I express heartfelt gratitude to my parents, **Barbara Friderike Hohlfeld and Dr. med. Siegfried Hohlfeld**, for their unwavering love, support, and encouragement throughout my life. Their belief in my potential has been instrumental in the pursuit of knowledge, innovation, and ideas.

The author also acknowledges the contributions of AI systems (ChatGPT by OpenAI, Gemini by Google) in refining the mathematical proof structures, generating supporting proofs, comparing textual versions of ideas and document structures, and helping to verify method correctness. All core ideas, concepts, and mathematical formulations are entirely the author's original creation.

References

- [1] Christian Heinrich Hohlfeld. *Hohlfeld Data Representation (HDR): A Deterministic and Proportional Framework for Data Encoding Using Residual Vectors*. December 7, 2024. (Author: Christian Heinrich Hohlfeld, Konstanz, Germany.)
- [2] Christian Heinrich Hohlfeld. *Rigorous Proof of Deterministic Division by Zero Method: Shift-Unshift*. 2024. (Author: Christian Heinrich Hohlfeld.)
- [3] Christian Heinrich Hohlfeld. Author website: christianhohlfeld.com. Accessed 2026-02-24.